UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

Echoes of Europa

Subject:	Temas Selectos de Ingeniería II
Group:	02
Professor.:	Dr. Sergio Teodoro Vite
Author(s).:	Chávez Villanueva Giovanni Salvador Cruz Cedillo Daniel Alejandro Nava Alberto Vanessa Tapia Estevez José Luis
Account numbers:	317003417 318263522 316083298

EVIDENCE SHEET

Echoes of Europa

LIST OF ACCOUNT NUMBER

317003417

318263522

316083298

SUMMARY

This final project aims to put into practice everything learned during the Selected Topics in Engineering II course, focusing on the simulation of basic physical behaviors and 3D modeling of various objects. In this project, we will explore the methodologies applied to different models, as well as simulations such as water, air, cloth movement, rigid objects, deformable objects, among others, using software like Unity and Blender.

The project is focused on providing an educational experience for users, consisting of an exploration of one of Jupiter's moons, Europa. This journey will be in first-person perspective, allowing users to encounter plants, rocks, educational information about the moon, among other elements, some of which will provide details about the objects or specific areas being explored on the moon. The journey is divided into four distinct zones with different characteristics: the first is a beach, the second a cave, the third a forest, and finally, a cliff where users can observe the entire map they explored.

INTRODUCTION

The planet Jupiter has 95 moons, one of the most popular being Europa. It is so well-known because its size, minerals, and ice content could be factors that allow for the possibility of life, similar to what exists on Earth.

> According to NASA (n.d.), Europa is almost 90% the size of Earth's Moon. Therefore, if we were to replace our Moon with Europa, it might appear to be the same size in the sky as our Moon, but much, much brighter.

So far, research indicates that Europa is a potentially habitable place, similar to Earth, as the moon contains minerals, water vapor, and an atmosphere suitable for Earth-like life. However, the moon's surface is smoother, as it has not experienced as many meteor impacts. Additionally, the entire surface is covered by a layer of ice approximately 15 to 25 km thick. It is worth noting that despite the ice, the surface has several brown-colored fractures caused by a combination of salts, sulfur, and water.

On the other hand, one of the fundamental branches of computer graphics is physics-based simulation, which allows us to recreate physical phenomena interactively and visually.

> According to (Teodoro-Vite et al., 2022), simulation is a set of processes aimed at representing possible scenarios or behaviors of a system over time.

In computer graphics, we have visual environments aimed at representing elements of the real world in a partial space. Additionally, we have the scene, which consists of the camera, the model, the lighting, and the interaction. To create a scene or a visual environment, we require models of the space we want to represent.

> According to (Teodoro-Vite et al. 2022), a model is an idealized representation of one or more properties of a structure, phenomenon, or system, emulating its shape, behavior, and/or characteristic traits to

physically and/or symbolically define the set of entities with the same or similar characteristics.

Teodoro Vite. (2022), “Introducción a la Simulación”. [PDF]. Recuperado el 13 de noviembre de 2024, de: https://drive.google.com/drive/u/1/folders/1DDwgUVyOqEdwbu1u0IY6tqR3Cc8nJS0o

Teodoro Vite. (2024), “Simulación de forma Modelado 3D”. [PDF]. Recuperado el 13 de noviembre de 2024, de: https://drive.google.com/file/d/1EFqlwhaw1kcufhErGgpedEy9qOsA1Fik/view?usp=sharing

Teodoro Vite. (2022), “Introducción a los gráficos por computadora”. [PDF]. Recuperado el 13 de noviembre de 2024, de: https://drive.google.com/file/d/1vowRGEPan-mn-aN6fySnDcnfHdbGwom9/view?usp=sharing

Nasa Ciencia. (2024). “¿Cuántas Lunas tiene cada planeta?”. [Artículo web]. Recuperado el 13 de noviembre de 2024, de: https://spaceplace.nasa.gov/how-many-moons/sp/

Nasa. (S/F). “Europa: Facts”. [Artículo Web]. Recuperado el 13 de noviembre de 2024, de: https://science.nasa.gov/jupiter/moons/europa/europa-facts/

METHODOLOGY

Models

For the main models, Blender software was used, applying various modeling techniques such as geometric modeling, hierarchical modeling, mechanical modeling, organic modeling, and physics-based modeling, which were learned throughout the course. It is worth noting that specific techniques were followed for each type of modeling, such as:

- Geometric, Description where we describe the model we want to recreate based on geometric shapes.
- Composition, which refers to the way the elements are organized.
- Jerarquía, es la relación estructural del objeto.
- Hierarchy, is the structural relationship of the object.
- Aspect Ratios, which describe the proportions of the model.
- Symmetry, which is the balanced and uniform distribution of elements along an axis or plane.
- Pose, is the position and orientation in which the body parts or elements of a model are placed.
- Repetitive Patterns, refer to the organized repetition of elements or shapes.

Let us remember that geometric modeling stands out for focusing on basic shapes, symmetry, and parameterization.

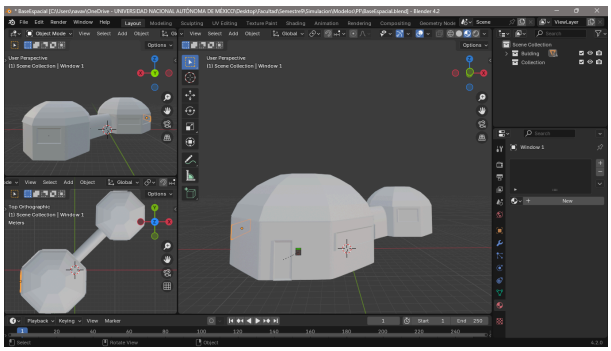

Figure 1: Space station using geometric modeling

Hierarchical modeling stands out as a combination of geometric, mechanical, and organic modeling, as it can be decomposed into geometric shapes, focusing on spatial relationships and complex forms.

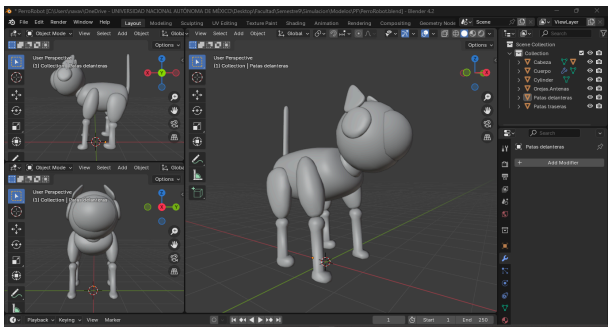

Figure 2: Robot dog using hierarchical modeling.

Mechanical modeling stands out because it focuses primarily on the functionality and spatial description of an object with a functional structure.

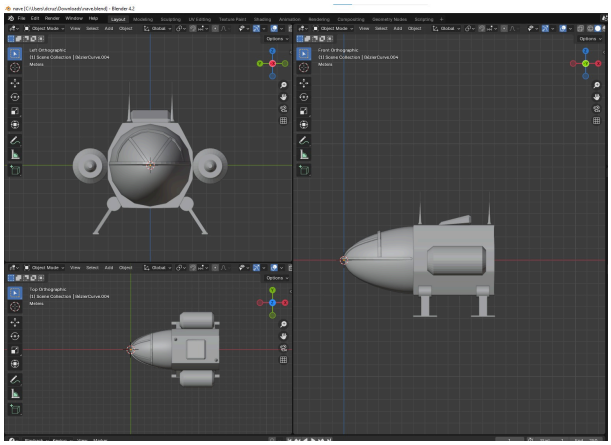

Figure 3: Spaceship using mechanical modeling..

Organic modeling stands out because it includes all models that cannot be represented with the previous methods, typically models from nature. It focuses on curves and complex shapes.

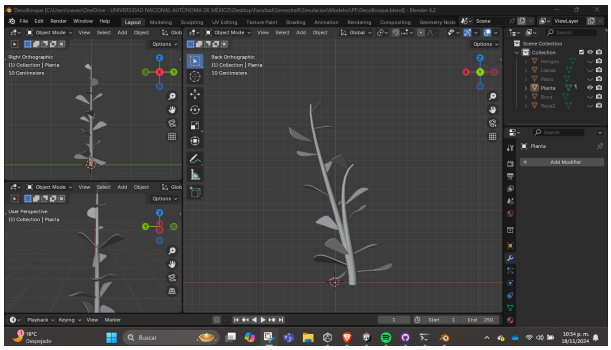

Figure 4: Plants using organic modeling.

Physics-based modeling stands out because geometric elements describe a fluidity that simulates movement.

Figure 5: Flag using physics-based modeling.

Blender tools such as modifiers (Array, Decimate, Mirror, Screw, Solidify, etc.), duplicate, parent objects, proportional editing, snap, among others, were used.

Figure 6: Decimate modifier.

Figure 7: Mirror modifier.

The Maya software was also used to create the blocking of the scene that the main character will traverse. This blocking and the idea for the journey were contributed by Tapia Estevez José Luis, a student of the Bachelor's Degree in Video Game Development and Programming at Coco School, who assisted in planning the moon's journey and creating the scene blocking.

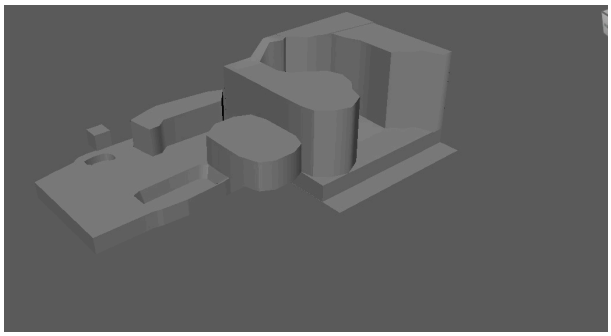

Figure 8: First zone of the scene: Beach.

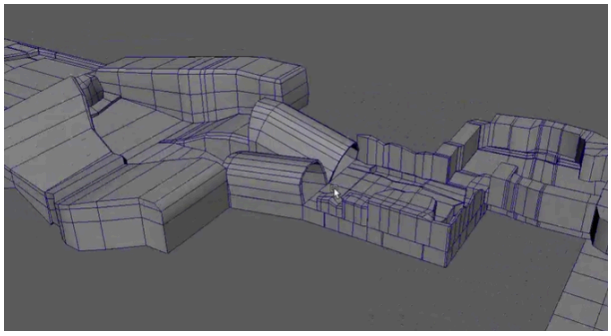

Figure 9: Second zone of the scene: Cave.

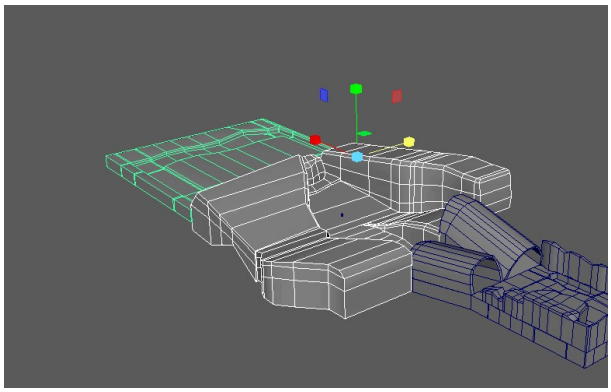

Figure 10: Third zone of the scene: Forest.

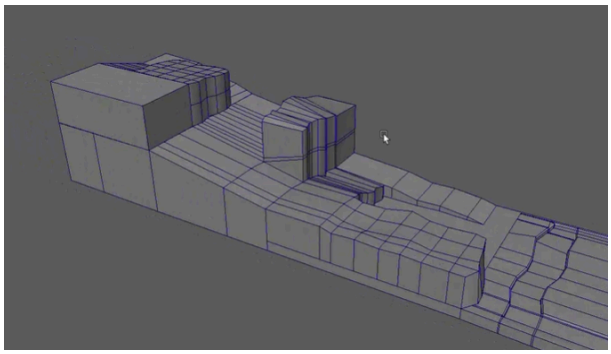

Figure 11: Fourth zone of the scene: Hike.

It is worth noting that work was done on these proposed scenes, but changes and modifications were made in Blender and Unity to give different volumes to each zone.

Simulations

For the simulations of physical behaviors, we used Unity software, where we were able to implement simulations such as cloth movement, rigid objects, and deformable objects using the NVIDIA Flex for Unity library in its 1.0 beta version.

We started by implementing water behavior in a pond. For this, a Flex container was used, which allowed us to create other ponds of different sizes along with an array-type asset to generate the water for them. The procedure implemented was the same as in the second term, as we needed the water to remain in a container.

However, it was necessary to make the water particles larger to ensure the game ran more smoothly.

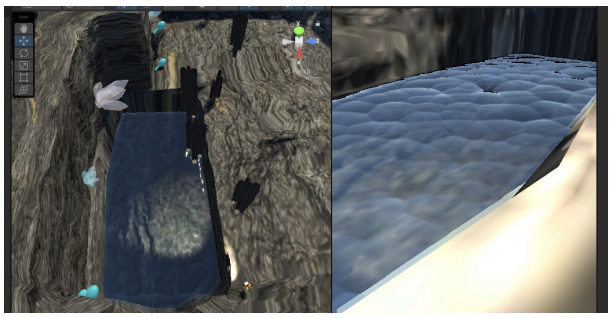

Figure 12: Simulation of water behavior in a pond.

We also added another water behavior: a droplet falling from a stalactite. For this, a Flex object of type source was created with a quad mesh. This mesh was stretched to display only two vertices in the scene, allowing the droplets to be visible in the game.

Figure 13: Falling droplet.

Next, we worked on cloth behavior, applying it to a flag, as we did in the second term. The same techniques used for the flags developed during the second term were applied. We once again used Flex containers and assets, but in this case, of type cloth, allowing us to simulate the movement of the flag.

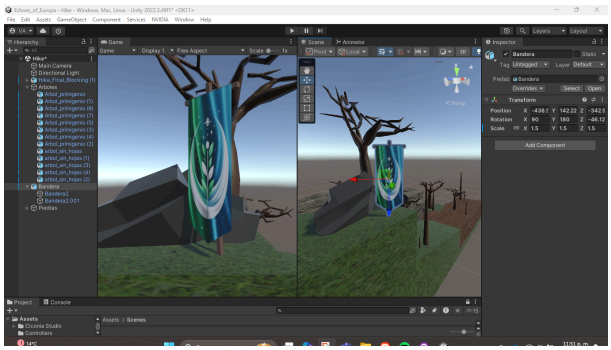

Figure 14: Flag added.

For the rigid object, we decided to implement it on a rock, to which a Flex solid asset was applied. This rock serves as an obstacle on the path, and when pushed, the deformation of mushrooms, which are soft bodies, can be observed. The container used for the flag was reused for this model, and the particle spacing of the rock was adjusted to reduce the number of particles needed.

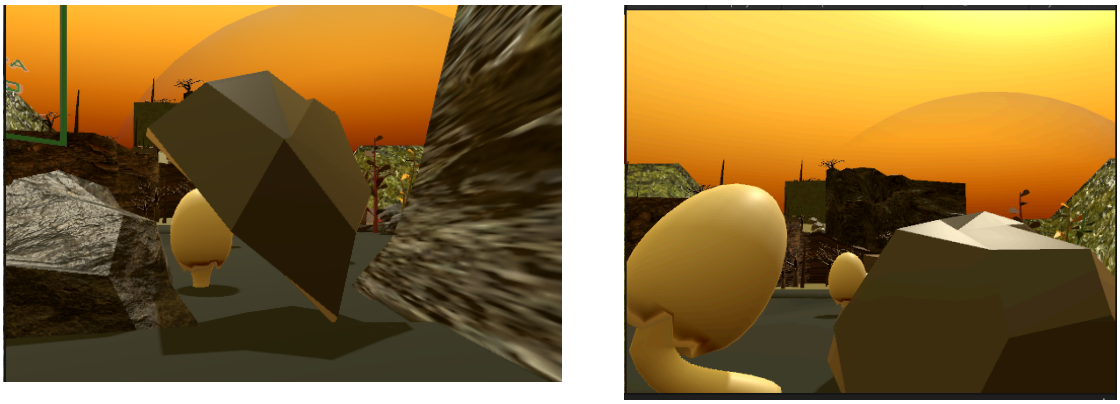

Figure 15: Rigid object.

As mentioned earlier, the soft body objects created were mushrooms in the scene, which deform upon contact with the player or a rock.

Figure 16: Deformable object.

Texturing and materials

To texture the modeled objects and the scene's environment, we decided to do it directly in Unity software.

Unity provides the option to create various types of materials, including plastics, metals, glass, matte colors, and the ability to use image-based textures. In our case, we chose to use all types of materials for our objects. Starting with the image-based textures, we obtained them from the following page:

https://ambientcg.com/

From that page, we decided to use the following textures.

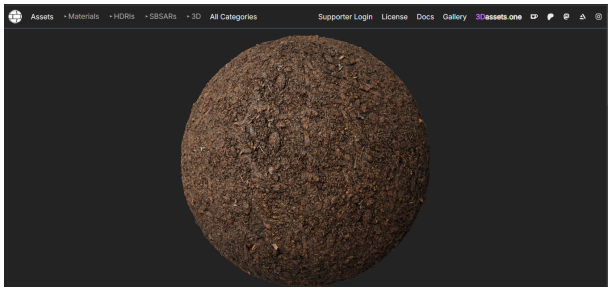

Figure 17: Soil texture. Retrieved on November 17, 2024, from: https://ambientcg.com/view?id=Ground048

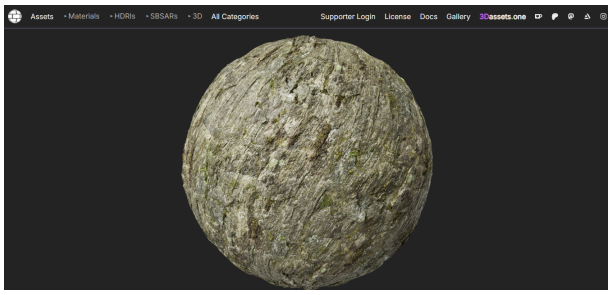

Figure 18: Mossy stone texture. Retrieved on November 17, 2024, from: https://ambientcg.com/view?id=Rock047

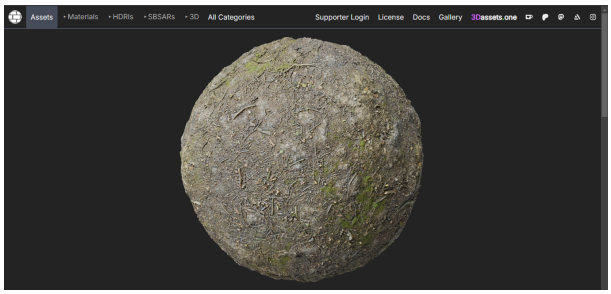

Figure 19: Soil texture. Retrieved on November 17, 2024, from: https://ambientcg.com/view?id=Ground024

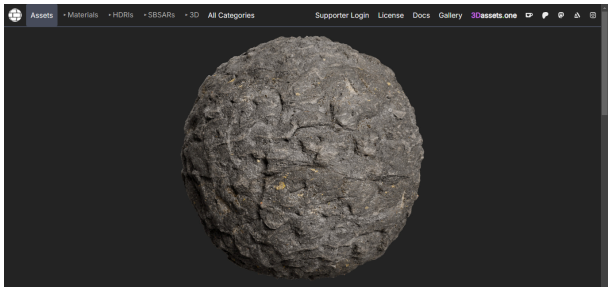

Figure 20: Stone texture. Retrieved on November 18, 2024, from: https://ambientcg.com/view?id=Rock050

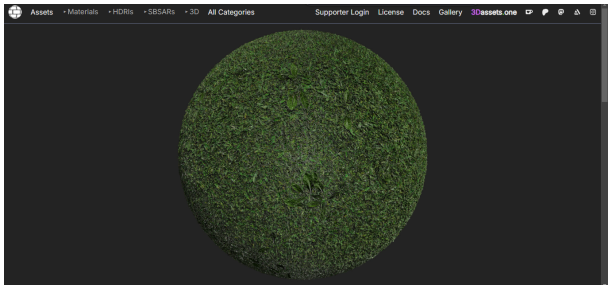

Figura 21: Grass texture. Retrieved on November 18, 2024, from: https://ambientcg.com/view?id=Grass002

Once the textures were downloaded and extracted from the .zip file, we copied them to our Textures folder, located within the Assets folder, where we were able to organize the textures to avoid confusion.

Figure 22: Folders included within the Textures folder.

Since each scene is different, we decided to create folders for each of them with their corresponding textures implemented within them.

Figure 23: Folders within the Forest folder.

Finally, we decided to recycle the textures across all the scenes, using textures from the forest in the cave, beach, and cliff, while adding extras or other textures as needed for the scenes.

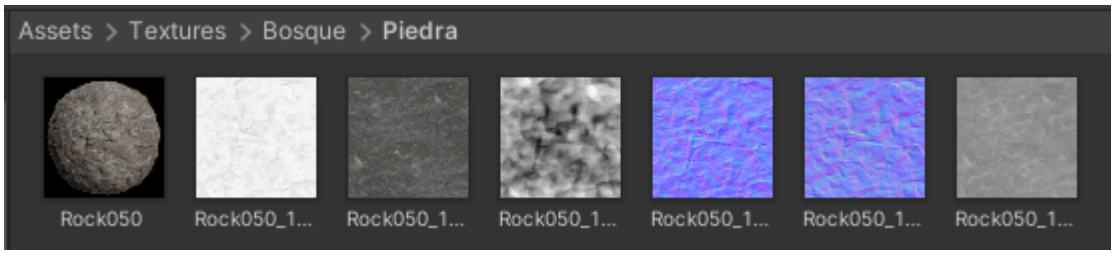

Figure 24: Elements within the Stone folder.

Each of these images was placed in the corresponding shader format.

We also used flat colors, as we decided to combine both techniques. If we had only used flat colors, the map would have appeared more cartoonish, but we decided to add some realism by combining these two texturing methods.

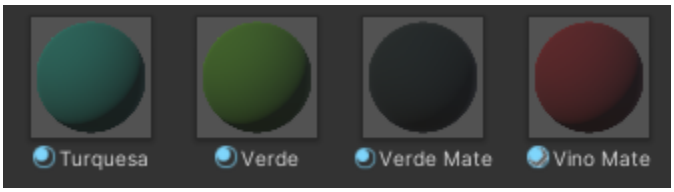

Figure 25: Some matte flat colors used for texturing.

Figure 26: Some glass-type colors used for texturing.

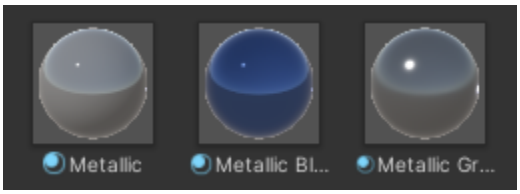

Figure 27: Some metallic-type colors used for texturing.

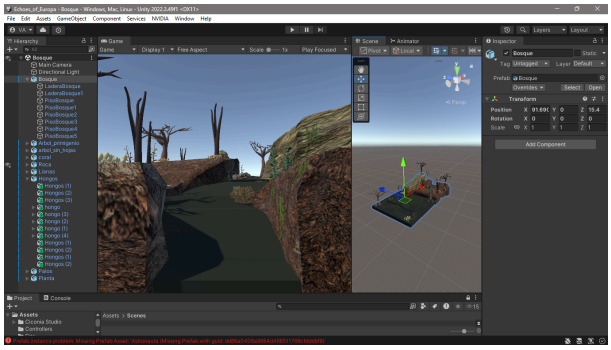

Figure 28: Combination of flat color textures and image-based textures.

Shaders

Two shaders were created. The first one allowed us to create the gradient effect for the planet Jupiter, and depending on whether it is day or night, this shader changes; it simulates the view of the moon from the planet Earth.

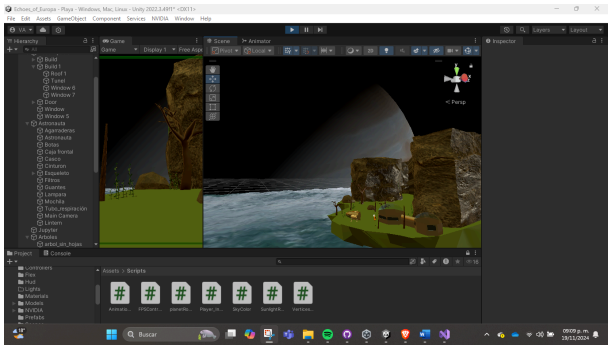

Figure 29: Gradient effect of the planet Jupiter.

```
Properties
{
    _MainTex ("Base Texture", 2D) = "white" {}
    //_SkyColor ("Sky Color", Color) = (0.5, 0.7, 1, 1)
}
SubShader
{
    Tags { "RenderType"="Transparent" }
    LOD 100

    Pass
    {
        Blend SrcAlpha OneMinusSrcAlpha   // Asegura la mezcla para transparencia
        ZWrite On
        ZTest LEqual
        CGPROGRAM
        #pragma vertex vert
        #pragma fragment frag
        #include "UnityCG.cginc"

        struct appdata
        {
            float4 vertex : POSITION;
            float3 normal : NORMAL;
            float2 uv : TEXCOORD0;
        };

        struct v2f
        {
            float4 pos : SV_POSITION;
            float2 uv : TEXCOORD0;
            float3 worldPos : TEXCOORD1;
            float3 normal : TEXCOORD2;
        };

        sampler2D _MainTex; //Textura principal
        float4 _MainTex_ST; //Control del desplazamiento y escala de la textura
```

Figure 30: Declaration of basic values for the shader.

To achieve the gradient effect in the image, a custom shader called *DistantPlanetShader* was created. This shader first takes the texture of the planet that was chosen to be used. Then, on the illuminated side, the texture is displayed, while on the dark side, it becomes transparent. Additionally, depending on the lighting stage, the colors are either orange, resembling sunrise, or darker hues.

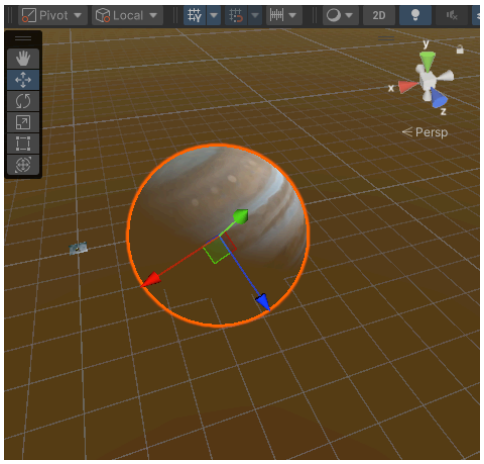

Figure 31: Model of the planet Jupiter.

```
// Colores para el cielo en diferentes momentos del día
float4 skyColorBlue = float4(0.5, 0.7, 1, 1);      // Azul
float4 skyColorOrange = float4(1.0, 0.5, 0.2, 1); // Naranja
float4 skyColorBlack = float4(0.0, 0.0, 0.0, 1);  // Negro

v2f vert (appdata v)
{
    v2f o;
    o.pos = UnityObjectToClipPos(v.vertex);
    o.normal = UnityObjectToWorldNormal(v.normal);
    o.worldPos = mul(unity_ObjectToWorld, v.vertex).xyz;
    o.uv = v.uv;

    return o;
}
```

Figure 32: Declaration of colors for day and night effect.

To achieve this effect, it started by establishing that the vertices obtain information about the light directed towards Jupiter from the Sunlight illumination, which rotates (its script will be explained later). Thanks to this rotation, the lights move, creating shadows on Jupiter and thus generating a day and night effect.

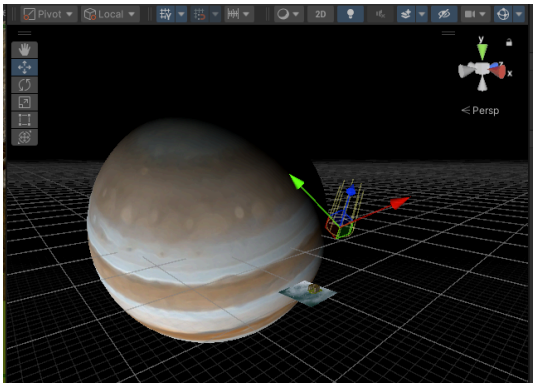

Figure 33: Sunlight illumination.

Once the vertices obtain the light information, they convert it into a dot product, which allows determining the direction of the light and identifying which side the light comes from. This enables the shader to illuminate the texture on the lit side and make the texture transparent on the dark side.

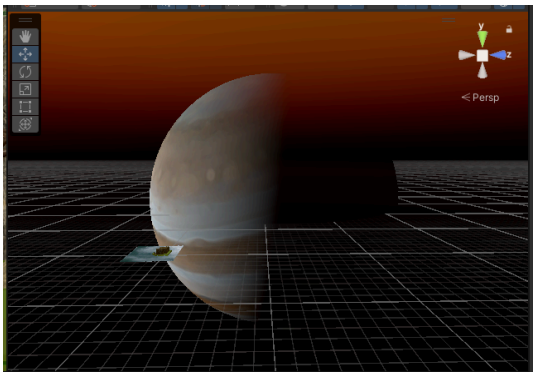

Figure 34: Transparency effect in the darkness.

```
float4 frag (v2f i) : SV_Target
{
    // Get texture Color
    float3 texColor = tex2D(_MainTex, i.uv).rgb;

    // Calculate light phase based on light direction
    float3 lightDir = normalize(_WorldSpaceLightPos0.xyz);
    float3 upDirection = float3(0.0, 1.0, 0.0);  // The "up" vector (for day/night detection)
    float dotProduct = dot(lightDir, upDirection);  // Dot product with the "up" vector

    // Determine sky color based on the light direction
    float3 interpolatedSkyColor;

    if (dotProduct > 0.5) // Midday (sun is high in the sky)
    {
        interpolatedSkyColor = skyColorBlue.rgb;
    }
    else if (dotProduct > 0.0) // Sunset or sunrise (sun near the horizon)
    {
        interpolatedSkyColor = skyColorOrange.rgb;
    }
    else // Night (sun is below the horizon)
    {
        interpolatedSkyColor = skyColorBlack.rgb;
    }

    // Blend the interpolated sky color with the texture based on light intensity
    float lightIntensity = saturate(dot(lightDir, i.normal)); // Light intensity based on the angle
    lightIntensity = pow(lightIntensity, 0.8);  // Optional: adjust light intensity contrast (smooths light transitions)

    // Instead of affecting the sky color with light intensity, only blend it
    float3 skyBlend = lerp(interpolatedSkyColor, texColor, lightIntensity);

    // Calculate alpha (transparency) based on light intensity
    float alpha = lightIntensity; // Areas with less light become more transparent

    // Return final color with calculated transparency
    return float4(skyBlend, alpha);
}
ENDCG
```

Figure 35: Instances to obtain the light direction and establish transparency in the dark area.

The second shader is the one that allowed us to create the effect of the ocean water. For this, a custom shader called *Ocean_waves* was created, which was used along with the *VerticesAnimation* script provided by the professor.

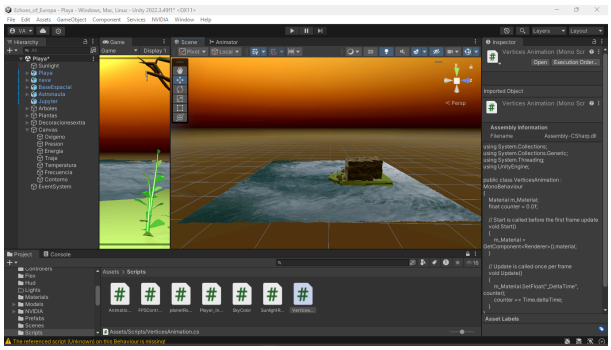

Figure 36: Plane-type mesh that will simulate the ocean with its applied texture.

For the *Ocean_waves* shader, modifications were made as seen in class. A *Delta time* variable was added, which allows us to modify the established mesh over time. Additionally, an equation was included where we set the force and direction in which we want the "waves" of the ocean to move and be simulated.

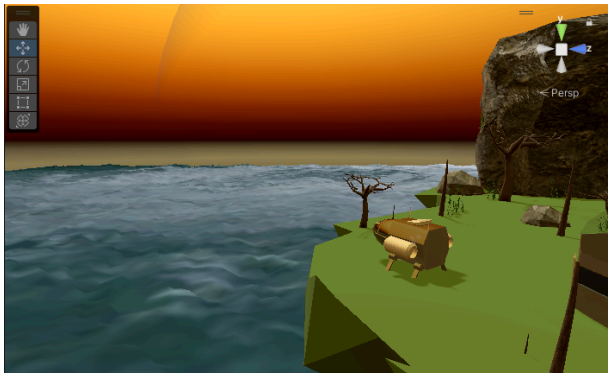

Figure 37: Simulation of ocean movement using sinusoidal equations.

```
sampler2D _MainTex;
float4 _MainTex_ST;
float _DeltaTime;

v2f vert (appdata v)
{
    v2f o;
    float randomOffset = Random(v.vertex.xyz)*0.01;
    //asociando ec matem*tica
    v.vertex.z += 0.005f*sin(10.0f*v.vertex.y + 2*_DeltaTime); //disminuir frecuencia reduce la fuerza
    v.vertex.z += 0.005*sin(20.0f*v.vertex.y + 0.1*_DeltaTime); //disminuir frecuencia reduce la fuerza
    //v.vertex.y += sin(v.vertex.y/10000 + _DeltaTime); //disminuir frecuencia reduce la fuerza
    v.vertex += randomOffset;
    o.vertex = UnityObjectToClipPos(v.vertex);
    o.uv = TRANSFORM_TEX(v.uv, _MainTex);
    UNITY_TRANSFER_FOG(o,o.vertex);
    return o;
}

fixed4 frag (v2f i) : SV_Target
{
    // sample the texture
    fixed4 col = tex2D(_MainTex, i.uv);
    // apply fog
    UNITY_APPLY_FOG(i.fogCoord, col);
    return col;
}
ENDCG
```

Figure 38: Modifications in the *Ocean_waves* shader.

Animations

Three animations were implemented. The first one is walking, where the user can navigate the world with the avatar walking in forward, backward, left, and right directions. For the implementation of this animation, the first step was to install a package from the Package Manager:

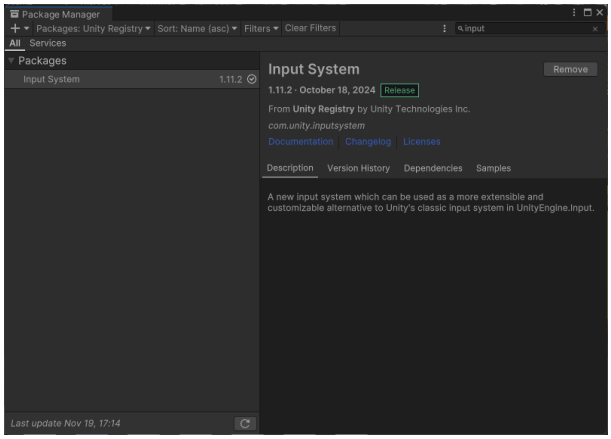

Figure 39: Installation of the Input System package.

This package will generate the following objects:

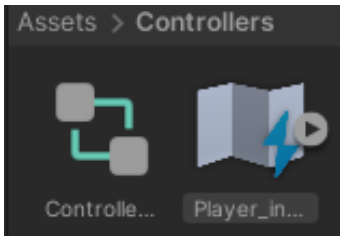

Figure 40: Objects generated by the package.

In which we establish which keys we want to implement for the animations or interactions we want to have.

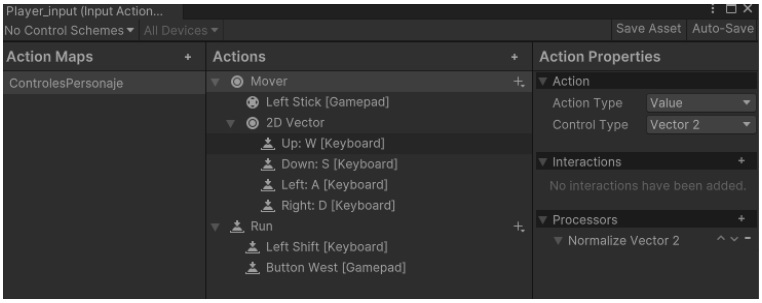

Figure 41: Setting the keys that are planned to be used.

In this case, we use the keys W, S, D, A to move forward, backward, right, and left, respectively. While holding Shift and pressing a movement key (A, W, S, D), the avatar will run. Finally, pressing the spacebar makes the avatar jump.

To capture the selection of specific keys, a C# class is created, which generates a script called *Player_Input*.

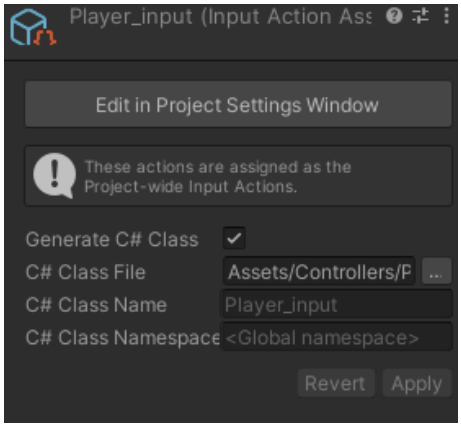

Figure 42: Activating the generation of the *Player_Input* script.

Thanks to this generated script, we can establish which keys will be associated with which actions.

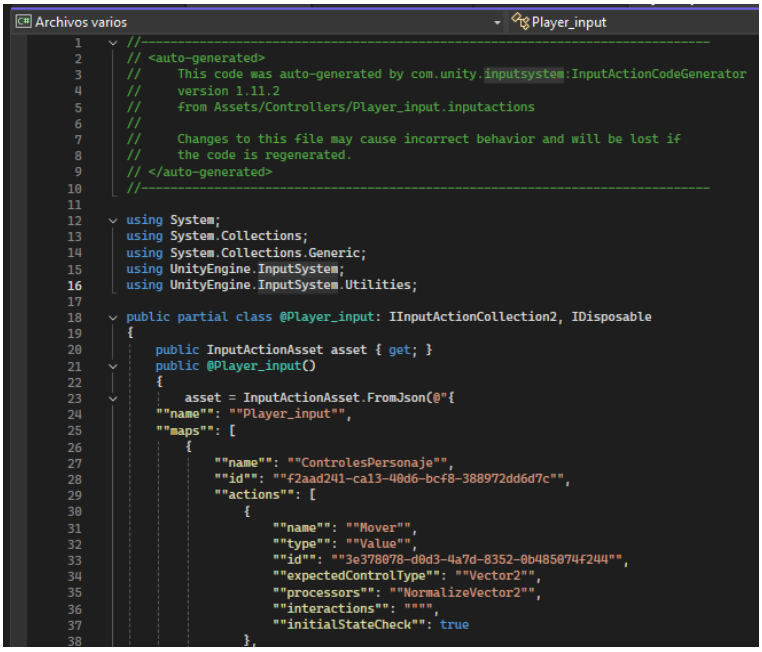

Figure 43: Script generado.

Key	Action.
A	Movement to the left.
W	Movement forward.
S	Movement backward.
D	Movement to the right.
F	Turning the flashlight on and off.
Spacebar	Jump.
Shift	Run.

Figure 44: Action table.

Now moving on to the *FPS Controller*, where we take the player's camera and the *flashlight* light, assigning these two objects to the avatar.

```
[RequireComponent(typeof(CharacterController))]
public class FPSController : MonoBehaviour
{
    public Camera playerCamera;

    public float lookSpeed = 2.0f;
    public float lookXLimit = 45.0f;
    public float jumpPower = 7f;

    public Light flashlight;
    float flashlightTiltSpeed = 2.0f; //Velocidad de inclinación
    float flashlightTiltLimit = 30.0f;

    public bool canMove = true;
    CharacterController characterController;
    Animator animator;

    Vector3 moveDirection = Vector3.zero;
    float rotationX = 0;
    float flashlightRotationX = 0;
    float gravity = 9.81f;

    float walkSpeed = 3.5f;
    float runSpeed =  7.0f;
```

Figure 45: Variables that determine movement speed, mouse sensitivity, etc.

Meanwhile, CharacterController allows us to directly associate keys with actions since the information is obtained and can be used without any issues.

```
// Start is called before the first frame update
void Start()
{
    animator = GetComponent<Animator>();
    characterController = GetComponent<CharacterController>();
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;

    if (flashlight != null)
        flashlight.enabled = false;
}

// Update is called once per frame
void Update()
{

    Handles Movment

    Handles Flashlight

    Handles Jumping


    Handles Rotation


    Update Animator States
}
```

Figure 46: Key associations with avatar actions.

For the implementation of the animations, we apply what was learned in class about the Animator.

```
#region Update Animator States
// Check if there's movement
bool isMoving = curSpeedX != 0 || curSpeedY != 0;

animator.SetBool("isWalking", isMoving && !isRunning);
animator.SetBool("isRunning", isMoving && isRunning);
#endregion
```

Figure 47: Animator statements that will allow us to change actions.

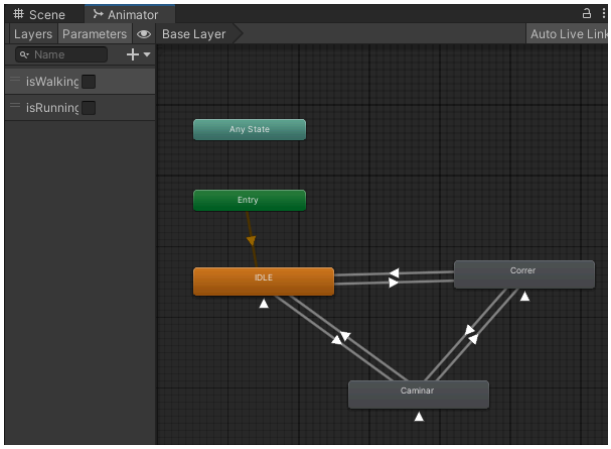

Figure 48: Animation cycle.

The animations used were obtained from: https://www.adobe.com/products/substance3d/plugins/mixamo-in-blender.html

Cameras

A first-person camera was implemented, which is anchored to the astronaut and allows us to follow the movement of the avatar, in this case, the astronaut. This avatar can walk forward, right, left, and backward, as we highlighted in the animation section.

Figure 49: First-person camera anchored to the astronaut.

To implement the first-person camera, it was only necessary to set it as a child of the astronaut model and anchor it to follow the movement of the mouse pointer. This was achieved in the *FPSController* script mentioned earlier.

```
#region Handles Rotation
characterController.Move(moveDirection * Time.deltaTime);

if (canMove)
{
    rotationX += -Input.GetAxis("Mouse Y") * lookSpeed;
    rotationX = Mathf.Clamp(rotationX, -lookXLimit, lookXLimit);
    playerCamera.transform.localRotation = Quaternion.Euler(rotationX, 0, 0);
    transform.rotation *= Quaternion.Euler(0, Input.GetAxis("Mouse X") * lookSpeed, 0);
}

#endregion
```

Figure 50: Camera and character rotation handling.

Lights

Two lights were implemented: a main light as the astronaut's *flashlight*, which illuminates the scene as the astronaut walks through it, and the *Sunlight* light, which allows us to create the day and night cycle in the scene.

The *flashlight* light is declared as an object and given the light property, so once it is anchored to the astronaut, it will follow their movements.

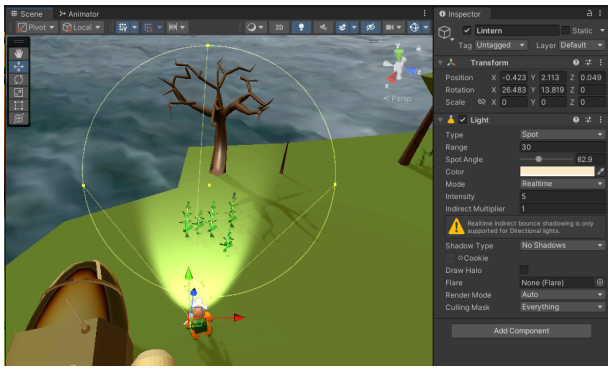

Figure 51: Spotlight, flashlight anchored to the astronaut.

For the *Sunlight* light, a rotation was implemented based on time:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class RotateDirectionalLight : MonoBehaviour
{
    public float angularSpeed = 10f;

    void Update()
    {
        transform.Rotate(Vector3.right * angularSpeed * Time.deltaTime);
    }
}
```

Figure 52: Rotation of the *Sunlight.*

Canvas

A UI element of the Canvas category is added, which will allow us to create a HUD.

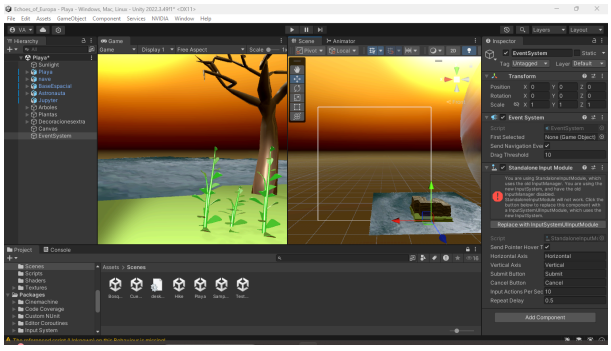

Figure 53: Adding the Canvas to the scene.

But first, it is important to establish that the HUD remains anchored to the first-person camera.

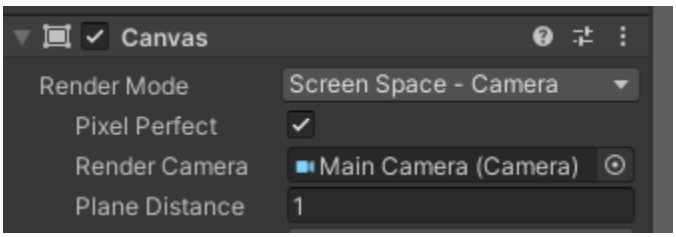

Figure 54: Canvas configuration.

We set the distance to 1 so that we can add the HUD elements to the astronaut's helmet.

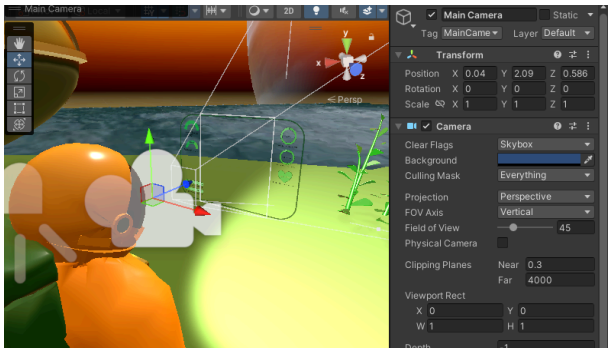

Figure 55: Canvas distance configuration.

We proceed to create the images that we want to appear in the avatar's HUD:

Figure 56: HUD information, part 1.

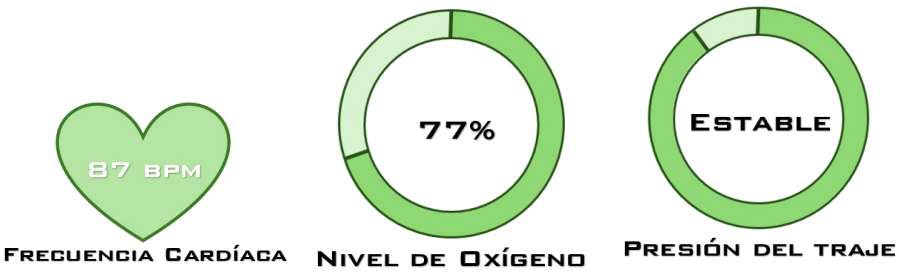

Figure 57: HUD information, part 2.

To add these images to the canvas, a **Canvas** type image must first be created. It is also necessary to add images with a transparent background so that only the figure is visible. For this, the following tool was used to remove the background: https://www.remove.bg/es/upload

Once the images with a transparent background were ready, we added them to the HUD folder.

Figure 58: Images with a transparent background.

To correctly add the images, the following modifications must be made to each one: first, set the Texture Type to Sprite (2D and UI) and set the Filter Mode to Point (no filter).

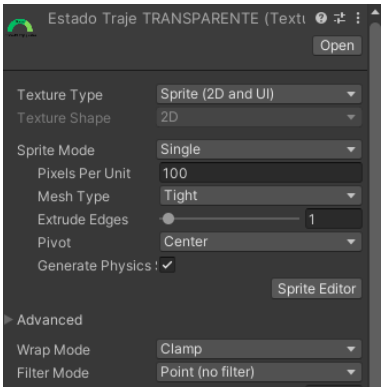

Figure 59: Modification to the canvas images.

Now, each modified image will be associated with its respective **Canvas Image**.

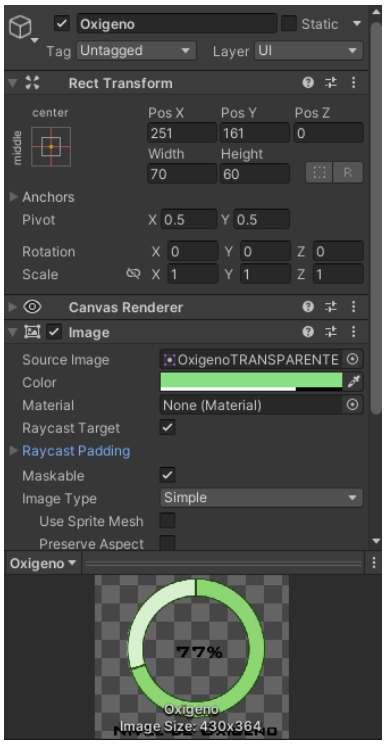

Figure 60: Association of the canvas with its respective image.

We added some color and transparency to give the HUD a screen effect, and we positioned each image in the canvas.

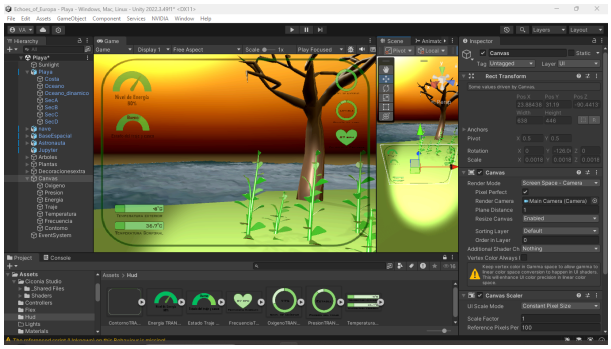

Figure 61: Positioning the astronaut's helmet HUD.

Scene Change

A transition with a canvas was also added to handle scene changes, featuring a loading screen.

Figure 62: Loading screen.

For this, Trigger-type colliders were used to modify the control variable of an animator, which in turn plays the animation. This animation was created by modifying the transparency values of two images over time.

Finally, the code that enables the scene change is triggered when the collider, as explained earlier, is contacted. The animation is then displayed, and the scene change occurs.

```
IEnumerator LoadLevel() {
    //Debug.Log("Se detecto colision");
    transitionAnim.SetTrigger("StartAnimation");
    yield return new WaitForSeconds(1.5f);
    SceneManager.LoadSceneAsync(SceneManager.GetActiveScene().buildIndex + 1);
    //transitionAnim.SetBool("StartAnimation", false);
}
```

Figure 63: Code for scene change.

To ensure that the HUD and this code do not disappear, the function was used is *DontDestroyOnLoad*.

```
    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }
```

]

Figure 63: Instance of a global helper object in the game.

RESULTS

Figure 64: Final result of the beach scene.

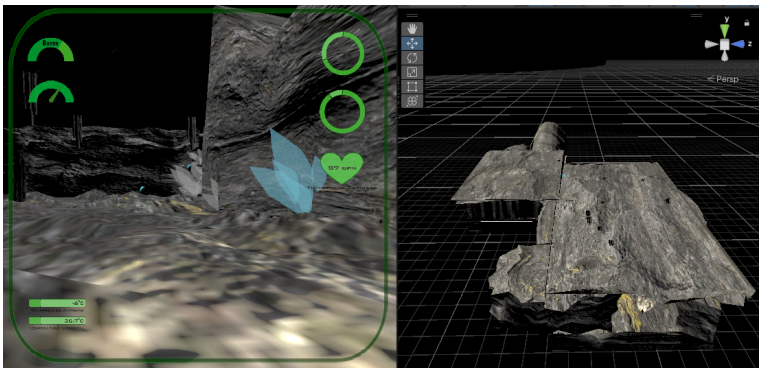

Figure 65: Final result of the cave scene.

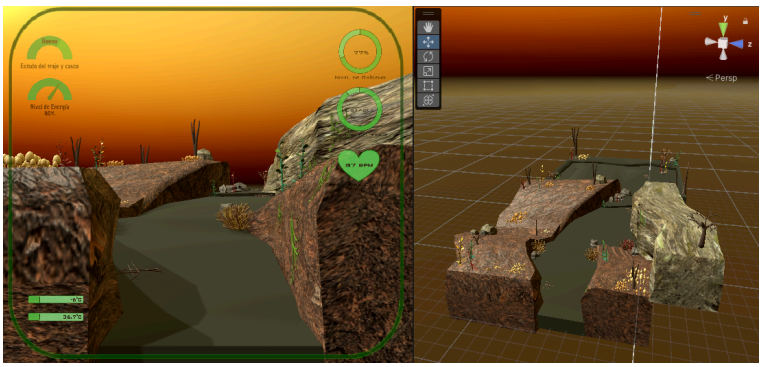

Figure 66: Final result of the forest scene.

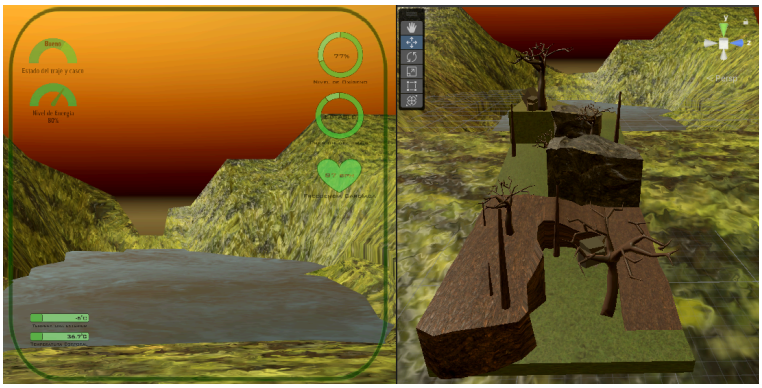

Figure 67: Final result of the hike scene.

REPOSITORY

https://github.com/gsalvador209/Echoes_of_Europa

CONCLUSIONS

Chávez Villanueva Giovanni Salvador

This project demonstrates the knowledge acquired throughout the course, where we represented different types of simulations by implementing a creative, interesting space with an entertaining proposal. While the final quality may not be of a high level, it is still worthy of representing what was learned in this course, as well as fulfilling its goal of educating and entertaining.

Cruz Cedillo Daniel Alejandro

We put into practice everything we learned in the course, from modeling with different techniques and tools provided by Blender, to adding animations and interactions made possible by Unity. We encountered several challenges along the way, which we improved upon as we advanced in the project, ultimately reaching the shown result. The knowledge gained today is a valuable tool that I believe will help us in future projects we participate in and create. It was a significant challenge to finish and achieve the results, but it truly helped us improve.

Nava Alberto Vanessa

It was possible to implement everything learned in the course. It was quite a challenging project, but we managed to showcase the results of our learning from each of the topics covered throughout the semester. I feel satisfied with what I learned and more eager to learn new things in Unity and Blender. I was quite surprised by how much I enjoyed the development of this project and how easy it became towards the end to implement various elements into it.

LINK TO THE VIDEO

//drive.google.com/file/d/1Y5p1qhulvTr1_AzgaMqIDnX1EYO-SuxD/view?usp=sharing

BIBLIOGRAPHIC REFERENCES

Teodoro Vite. (2022), “Introducción a la Simulación”. [PDF]. Recuperado el 13 de noviembre de 2024, de: https://drive.google.com/drive/u/1/folders/1DDwgUVyOqEdwbu1u0IY6tqR3Cc8nJS0o

Teodoro Vite. (2024), “Simulación de forma Modelado 3D”. [PDF]. Recuperado el 13 de noviembre de 2024, de: https://drive.google.com/file/d/1EFqlwhaw1kcufhErGgpedEy9qOsA1Fik/view?usp=sharing

Teodoro Vite. (2022), “Introducción a los gráficos por computadora”. [PDF]. Recuperado el 13 de noviembre de 2024, de: https://drive.google.com/file/d/1vowRGEPan-mn-aN6fySnDcnfHdbGwom9/view?usp=sharing

Nasa Ciencia. (2024). “¿Cuántas Lunas tiene cada planeta?”. [Artículo web]. Recuperado el 13 de noviembre de 2024, de: https://spaceplace.nasa.gov/how-many-moons/sp/

Nasa. (S/F). “Europa: Facts”. [Artículo Web]. Recuperado el 13 de noviembre de 2024, de: https://science.nasa.gov/jupiter/moons/europa/europa-facts/

Colaboradores de El Universal. (2019). “Visita y toca las rocas lunares que se exhiben en México”. Recuperado el 18 de noviembre, de: https://www.eluniversal.com.mx/ciencia-y-salud/visita-y-toca-las-piedras-lunares-que-se-exhiben-en-mexico/

Colaboradores de AstroIngeo.com. (2024). “Distancia entre el Sol y Júpiter”. Recuperado el 18 de noviembre, de: https://blog.astroingeo.org/sistema-solar/distancia-entre-el-sol-y-jupiter